# CPEI Documentation

Seo Bin (Selene) Han

April 9, 2026

# Contents


**1 Code Overview** **2**

**2 Function 1: `run_RFB`** **3**

**3 Function 2: `compute_cpei`** **6**

**4 Function 3: `rps_peak`** **6**

**5 Functions 4–5: Welch SNR and Peak** **7**

**6 Function 6: `compute_PLV`** **7**

**7 Function 7: `compute_ITPC`** **8**

**8 Function 8: `yin_estimate`** **9**

**9 Synthetic Validation** **11**

9.1 Signal 1: Clean Harmonic FFR . . . . . 11

9.2 Signal 2: Attenuated Fundamental . . . . . 12

9.3 Signal 3: Drifting $F_0$ . . . . . 13

9.4 Signal 4: Pure Noise . . . . . 15

9.5 Signal 5: Overlapping Harmonics . . . . . 16

**10 Final Algorithm Comparison** **17**

## Code Overview

The script `Synthetic_Validation.m` is organized into eight functions and a main script body. I passed all parameters explicitly to each function as arguments rather than using global variables. I did this because it made each function easier to test on its own, and it avoided a scoping bug in MATLAB where nested functions can cause variables to print to the terminal unexpectedly.

```
fs         = 8192;
dur        = 5;
t          = (0:1/fs:dur-1/fs).';
N          = length(t);

n_harm     = 8;
F0         = 120;

% welch parameters
win_sec    = 2;
nfft_w     = 2^nextpow2(win_sec * fs);
noverlap   = floor(nfft_w / 2);
win_w      = hann(nfft_w, 'periodic');

% RFB search range: 50 to 500 Hz
fmin_s     = 50;
fmax_s     = 500;
Pmin       = max(2, floor(fs/fmax_s));
Pmax       = ceil(fs/fmin_s);
periods    = Pmin:Pmax;
period_hz = fs ./ periods;

% CPEI target bandwidth
cpei_bw    = 5;

% YIN parameters
yin_thresh  = 0.1;
yin_win_ms  = 25;
yin_win     = round(yin_win_ms / 1000 * fs);
yin_fmin    = 50;
yin_fmax    = 500;
```

Listing 1: Shared parameters set at the top of the script

**Why I Made This Choice**

A few notes on why I picked these values:

- **fs = 8192 Hz** I matched this to the click ABR dataset so the synthetic results are directly comparable.
- **F0 = 120 Hz** This sits in the middle of the typical male voice range (roughly 85–180 Hz), so it is a reasonable representative target.
- **Search range 50–500 Hz** This covers the full human $F_0$ range with some margin on both sides.
- **Hann window** This is the standard window for spectral estimation. It reduces spectral leakage (energy "bleeding" between nearby frequency bins) compared to a plain rectangular window.
- **YIN window = 25 ms** At $F_0 = 120$ Hz, one pitch cycle takes about 8.3 ms. So 25 ms captures roughly 3 full cycles, which is the minimum needed to reliably detect the period.

## Function 1: `run_RFB`

This function builds a Ramanujan filter for each candidate period $p$ and returns a single number representing the average energy of the signal at that period. I like to interpret it as a bank of tuned detectors, one detector per possible pitch period (and each one reporting how much of the signal matches that rhythm).

```
function mean_E = run_RFB(signal, fs, fmin, fmax)
    % build the list of candidate periods to test
    Pmin_l = max(2, floor(fs/fmax));
    Pmax_l = ceil(fs/fmin);
    plist  = Pmin_l:Pmax_l;

    Rcq = 7;   % how many times to repeat the Ramanujan filter
    Rav = 3;   % how many times to repeat the averaging filter

    % clean up the signal before processing
    x   = signal(:);
    x   = x - mean(x);
```

```
    if max(abs(x)) > 0
        x = x ./ max(abs(x));
    end

    mean_E = zeros(1, length(plist));

    for idx = 1:length(plist)
        p      = plist(idx);

        % find which integers from 1 to p share no common factor
           with p
        k_orig = 1:p;
        k      = k_orig(gcd(k_orig, p) == 1);

        % compute one period of the Ramanujan sum
        cq = zeros(p, 1);
        for n = 0:(p-1)
            for a = k
                cq(n+1) = cq(n+1) + exp(1j*2*pi*a*n/p);
            end
        end
        cq = real(cq);

        % build the Ramanujan filter by repeating and normalizing
        FR  = repmat(cq, Rcq, 1);
        FR  = FR ./ norm(FR);

        % build a simple averaging (smoothing) filter
        FA  = repmat(ones(p,1), Rav, 1);
        FA  = FA ./ length(FA);

        % apply filter, square to get energy, smooth, store mean
        y   = conv(x, FR, 'same');
        y   = abs(y).^2;
        y   = conv(y, FA, 'same');
        mean_E(idx) = mean(y);
    end
end
```

Listing 2: `run_RFB` code

`Pmin_l = max(2, floor(fs/fmax))` finds the shortest period in samples that corresponds to the highest frequency we want to search. I enforced a minimum of 2 because a period of 1 sample would just be a DC (constant) signal, which is not useful. `Pmax_l = ceil(fs/fmin)` gives the longest period we need to test.

I then subtracted the mean to remove any DC offset. This matters because the Ramanujan filter at any period $p$ has zero mean by construction, so a DC offset in the signal would create spurious energy at every period. I also normalized to unit peak amplitude so that energy values are comparable across signals of different sizes.

`gcd(k_orig, p) == 1` picks out the integers between 1 and $p$ that share no common factor with $p$. These are the indices used in the Ramanujan sum formula. For example, for $p = 6$ the coprime indices are just $\{1, 5\}$, because 2, 3, and 4 all share factors with 6. For a prime $p$ like 7, every integer from 1 to 6 is coprime with it.

The double loop builds one period of the Ramanujan sequence $c_p(n)$ directly from the definition. The imaginary part is removed at the end because it cancels out mathematically the numerical residue that MATLAB will usually leave behind is claened up by `real()`.

`repmat(cq, Rcq, 1)` repeats the one-period sequence 7 times, making a longer filter. Longer filters have sharper frequency tuning but take more computation. I kept `Rcq = 7` from the original codebase. Dividing by `norm(FR)` makes the filter unit-energy so that output values are comparable across periods of different lengths.

The boxcar filter (a flat sequence of ones) repeated 3 times and normalized is just a local time-average. It smooths out the moment-to-moment variation in energy, giving a more stable estimate.

`conv(x, FR, 'same')` runs the signal through the Ramanujan filter. Squaring the magnitude gives a non-negative energy signal. Convolving with `FA` smooths it over time. Taking the mean collapses everything down to one number: the average energy at this period across the whole signal.

**Why I Made This Choice**

I kept `Rcq = 7` and `Rav = 3` from the original codebase because the synthetic results looked reasonable. Trying different values to see if performance improves is something I would want to do in future work.

## Function 2: compute_cpei

```
function cpei = compute_cpei(mean_E, period_hz, target_hz, bw_hz)
    % select bins within the target frequency band
    mask = abs(period_hz - target_hz) <= bw_hz;
    if sum(mask) == 0
        cpei = 0;
        return;
    end
    % CPEI = fraction of total energy in the target band
    cpei = sum(mean_E(mask)) / (sum(mean_E) + eps);
end
```

Listing 3: compute_cpei code

This function takes the energy profile from run_RFB and produces a single number between 0 and 1.

mask selects all period bins within ±bw_hz of the target. I used ±5 Hz for steady signals and ±30 Hz for the drifting pitch signal. The ±5 Hz window is narrow enough to exclude harmonics but wide enough to catch the ≈ 2 Hz offset caused by integer period rounding. The ±30 Hz window for the drifting signal is deliberately wide because the pitch sweeps from 100 to 150 Hz and we need to capture the full range.

eps in the denominator just prevents a divide-by-zero crash if the signal is silent. Dividing by total energy (rather than total signal power) means the CPEI does not depend on how loud the signal is, i.e. two people with the same neural tracking quality but different EEG amplitudes will get the same CPEI score.

## Function 3: rps_peak

```
function pk = rps_peak(mean_E, period_hz, fmin, fmax)
    mask    = period_hz >= fmin & period_hz <= fmax;
    E_sub   = mean_E(mask);
    f_sub   = period_hz(mask);
    [~,idx] = max(E_sub);
    pk      = f_sub(idx);
end
```

Listing 4: rps_peak code

This function just finds the frequency where the RFB energy is highest within a given range. While `compute_cpei` asks "how much of the total energy is near my target?", this asks "where is the energy peak?" These two can give different answers (e.g. in Signal 2, the CPEI score stayed above the noise floor even after the peak frequency had shifted to the wrong octave.)

## Functions 4–5: Welch SNR and Peak

```
function snr_db = welch_snr(psd, f, target, sbw, nlo, nhi)
    sm = abs(f - target) <= sbw;
    nm = (f >= nlo(1) & f <= nlo(2)) | (f >= nhi(1) & f <= nhi(2)
       );
    if sum(sm)==0 || sum(nm)==0
        snr_db = NaN;
        return;
    end
    snr_db = 10*log10(mean(psd(sm)) / mean(psd(nm)));
end

function pk = welch_peak(psd, f, fmin, fmax)
    mask    = f >= fmin & f <= fmax;
    [~,idx] = max(psd(mask));
    f_sub   = f(mask);
    pk      = f_sub(idx);
end
```

Listing 5: Welch SNR and peak frequency helpers

For Welch, I compared the average PSD in a narrow band around the target frequency against the average PSD in two flanking noise bands on either side. The ratio of those two averages, converted to decibels, is the SNR.

I used two flanking bands rather than one (to avoid bias). If the overall spectrum has a slope, using just one side would make the noise estimate too high or too low. Two symmetric bands cancel this effect out. The bands (`nlo` = [100, 115] Hz and `nhi` = [125, 140] Hz for $F_0 = 120$ Hz) were placed close to the target but far enough away to avoid the peak itself and the first harmonic at 240 Hz.

## Function 6: `compute_PLV`

```
function plv = compute_PLV(signal, fs, target_hz)
    t_loc   = (0:length(signal)-1).' / fs;
    ref     = exp(1j * 2*pi * target_hz * t_loc);
    x_hilb  = hilbert(signal(:));
    phase_x = angle(x_hilb);
    phase_r = angle(ref);
    plv     = abs(mean(exp(1j * (phase_x - phase_r))));
end
```

Listing 6: `compute_PLV` code

PLV measures how consistently the signal stays in sync with a reference oscillator ticking at `target_hz`. At every time sample, I calculated the phase difference between the signal and the reference and represented it as a point on the unit circle in the complex plane. If the signal is reliably locked to the reference, all those points cluster in one direction and their average has a large magnitude (close to 1). If the phase relationship is random, the points scatter in all directions and the average cancels out to near zero. PLV is therefore always between 0 and 1:

$$\mathrm{PLV} = \left| \frac{1}{N} \sum_{n=0}^{N-1} e^{j(\phi_x[n] - \phi_{\mathrm{ref}}[n])} \right|. \tag{1}$$

The Hilbert transform `hilbert(signal)` gives the instantaneous phase of the signal at every time sample. The `angle()` function then extracts that phase.

**Why I Made This Choice**

An alternative way to compute PLV is to use the Fourier transform instead of the Hilbert transform. That approach is more efficient for epoched data. I went with the Hilbert approach here because the signals are continuous (not epoched), and it felt more natural to track the instantaneous phase moment-by-moment through the signal.

## Function 7: `compute_ITPC`

```
function itpc = compute_ITPC(epochs, fs, target_hz)
    [n_ep, n_samp] = size(epochs);
    t_loc = (0:n_samp-1) / fs;
    ref   = exp(1j * 2*pi * target_hz * t_loc);

```

```
    phase_mat = zeros(n_ep, n_samp);
    for e = 1:n_ep
        x_hilb         = hilbert(epochs(e,:));
        phase_mat(e,:) = angle(x_hilb);
    end

    phase_ref = angle(repmat(ref, n_ep, 1));
    itpc      = abs(mean(exp(1j * (phase_mat - phase_ref)), 1));
    itpc      = mean(itpc);
end
```

Listing 7: `compute_ITPC` code

ITPC and PLV measure the same underlying thing, phase consistency with a reference, but they do it in different directions. PLV looks across time within one signal. ITPC looks across repeated trials (epochs) at each time point, then averages the result over time.

For the synthetic validation I built epochs by cutting each 5-second signal into non-overlapping 50 ms chunks:

```
epoch_ms  = 50;
epoch_len = round(epoch_ms / 1000 * fs);
n_epochs  = floor(N / epoch_len);
```

This gave 99 epochs per signal, each long enough to contain about 6 full pitch cycles at $F_0 = 120$ Hz. I chose 50 ms as a balance between having enough samples per epoch for reliable phase estimation and having enough epochs to get a stable average.

## Function 8: `yin_estimate`

```
function [f0_est, confidence] = yin_estimate(signal, fs, ...
                                  fmin, fmax, win_len, thresh)
    x    = signal(:);
    tmin = floor(fs/fmax);
    tmax = floor(fs/fmin);
    tmax = min(tmax, floor(win_len/2));

    % step 1: squared difference function
    d = zeros(tmax, 1);
    for tau = 1:tmax
```

```
        seg1   = x(1:win_len-tau);
        seg2   = x(1+tau:win_len);
        d(tau) = sum((seg1 - seg2).^2);
    end

    % step 2: cumulative mean normalized difference function
    d_prime    = zeros(tmax, 1);
    d_prime(1) = 1;
    running    = 0;
    for tau = 2:tmax
        running      = running + d(tau);
        d_prime(tau) = d(tau) / (running / tau);
    end

    % step 3: find first minimum below threshold
    f0_est     = NaN;
    confidence = 0;
    for tau = tmin+1:tmax-1
        if d_prime(tau) < thresh && ...
           d_prime(tau) < d_prime(tau-1) && ...
           d_prime(tau) <= d_prime(tau+1)

            % step 4: parabolic interpolation for sub-sample accuracy
            denom = d_prime(tau-1) - 2*d_prime(tau) + d_prime(tau+1);
            if abs(denom) > eps
                tau_fine = tau + 0.5 * ...
                    (d_prime(tau-1) - d_prime(tau+1)) / denom;
            else
                tau_fine = tau;
            end

            f0_est     = fs / tau_fine;
            confidence = 1 - d_prime(tau);
            break;
        end
    end
```

```
end
```

Listing 8: `yin_estimate` code

YIN is a pitch detection algorithm. Unlike CPEI or PLV, it does not measure how strong a rhythm is but rather it estimates what the pitch frequency actually is.

## Synthetic Validation

I tested all five algorithms on five different synthetic signals, each designed to test a specific property of a good neural entrainment measure. All signals were 5 seconds long at 8192 Hz.

### *Signal 1: Clean Harmonic FFR*

```
clean1 = zeros(N, 1);
for k = 1:n_harm
    clean1 = clean1 + (1/k) * sin(2*pi*k*F0*t);
end
clean1 = clean1 / max(abs(clean1));

for k = 1:length(snr_levels)
    snr_db = snr_levels(k);
    np     = 10^(-snr_db/20);
    noisy  = clean1 + np * randn(N, 1);
end
```

Listing 9: Signal 1 generation

This signal is a clean harmonic sound at $F_0 = 120$ Hz with 8 overtones, where each overtone is weaker than the one below it (amplitude $= 1/h$ for harmonic $h$). It's supposed to match the spectral shape of a real voiced vowel. I added white noise at five different levels to test how each algorithm degrades as the signal gets harder to detect.

The noise amplitude formula `np = 10^(-snr_db/20)` comes from the definition of SNR in decibels: $\text{SNR} = 20\log_{10}(A_{\text{signal}}/A_{\text{noise}})$.

Table 1: Signal 1 results: Sensitivity at varying SNR

| SNR (dB) | CPEI | Welch (dB) | PLV | ITPC | YIN (Hz) | YIN conf. |
|---|---|---|---|---|---|---|
| +20 | 0.316 | 38.40 | 0.901 | 0.513 | 120.0 | 0.966 |
| +10 | 0.255 | 28.47 | 0.723 | 0.402 | NaN | — |
| 0 | 0.109 | 18.09 | 0.343 | 0.195 | NaN | — |
| −10 | 0.049 | 8.625 | 0.111 | 0.102 | NaN | — |
| −20 | 0.041 | 2.818 | 0.033 | 0.084 | NaN | — |

Noise floors: CPEI = 0.040, PLV = 0.006, ITPC = 0.089.

**What the Results Mean**

PLV, ITPC, and Welch all stayed detectable down to −10 dB. CPEI held on until 0 dB, then fell to its noise floor. YIN only worked at the cleanest level (+20 dB). On a clean steady signal, CPEI is actually less sensitive than the other methods and I think that is fine, because CPEI's real advantages show up in the more difficult conditions tested in Signals 2 and 3.

***Signal 2: Attenuated Fundamental***

```
fixed_np = 0.3;
for k = 1:length(alpha_levels)
    alpha  = alpha_levels(k);
    sig2   = alpha * sin(2*pi*F0*t);
    for h = 2:n_harm
        sig2 = sig2 + (1/h)*sin(2*pi*h*F0*t);
    end
    sig2   = sig2 / max(abs(sig2));
    noisy2 = sig2 + fixed_np * randn(N, 1);

    wpk2   = welch_peak(p2, f2, fmin_s, fmax_s);
    werr2  = abs(wpk2 - 2*F0) < abs(wpk2 - F0);
end
```

Listing 10: Signal 2 generation

This signal simulates what happens in hearing loss: the brain's response at the base pitch ($F_0$) gets weaker while the overtones ($2F_0$, $3F_0$, etc.) stay relatively intact. I controlled this

by multiplying the fundamental component by a factor `alpha` ranging from 1.0 (full strength) down to 0.05 (5% of original).

The octave error check `abs(wpk2 - 2*F0) < abs(wpk2 - F0)` asks: is the Welch peak closer to 240 Hz than to 120 Hz? If yes, Welch has made an octave error.

Table 2: Signal 2 results: Octave error under fundamental attenuation

| $\alpha$ | CPEI | W.SNR (dB) | PLV | ITPC | RPS pk (Hz) | W.Err? |
|---|---|---|---|---|---|---|
| 1.00 | 0.262 | 28.63 | 0.741 | 0.408 | 117.0 | No |
| 0.50 | 0.165 | 23.86 | 0.425 | 0.260 | 120.5 | No |
| 0.20 | 0.102 | 16.46 | 0.100 | 0.153 | 240.9 | **Yes** |
| 0.10 | 0.091 | 11.69 | 0.013 | 0.133 | 240.9 | **Yes** |
| 0.05 | 0.089 | 6.542 | 0.074 | 0.127 | 240.9 | **Yes** |

RPS octave errors: 0/5. Welch: 3/5. YIN: 0 errors (but produced no estimates).

**What the Results Mean**

Welch made octave errors in 3 out of 5 conditions, ie. as soon as the fundamental dropped below 20% strength, Welch jumped to reporting 240 Hz instead of 120 Hz. RPS made zero octave errors across all five conditions. Yay!

I think it is important to note that at $\alpha \leq 0.20$ the RPS peak frequency also shifted to 240.9 Hz. But the CPEI *scalar* stayed above the noise floor (0.089 vs. noise floor 0.040) even at $\alpha = 0.05$. Hence the scalar is saying that "there is still real periodic energy near 120 Hz" even when the peak of the profile has moved. CPEI and `rps_peak` are answering different questions, and in this condition the scalar is more clinically useful than the peak.

***Signal 3: Drifting $F_0$***

```
F0_start = 100;   F0_end = 150;
F0_t     = linspace(F0_start, F0_end, N).';

% cumsum gives the instantaneous phase of a frequency-modulated signal
phase3 = 2*pi * cumsum(F0_t) / fs;

sig3 = zeros(N, 1);
```

```
for h = 1:6
    sig3 = sig3 + (1/h) * sin(h * phase3);
end
sig3   = sig3 / max(abs(sig3));
noisy3 = sig3 + 0.3 * randn(N, 1);

F0_mid = (F0_start + F0_end) / 2;    % = 125 Hz
cpei3  = compute_cpei(mE3, period_hz, F0_mid, 30);
```

Listing 11: Signal 3 generation

As mentioned in our meeting, real speakers don't maintain a perfectly constant pitch. They glide up and down naturally throughout an utterance. I used this signal to simulate that by sweeping $F_0$ linearly from 100 to 150 Hz over 5 seconds.

The phase is calculated using `cumsum(F0_t) / fs` rather than simply `F0_t .* t`. Although I initially used `F0_t .* t`, I noticed it would give wrong phase accumulation and not produce the intended sweep.

Table 3: Signal 3 results: Drifting $F_0$ detection

| **Algorithm** | **Output** | **In range?** |
|---|---|---|
| CPEI (±30 Hz pool) | 0.466 | — |
| RPS peak | 136.5 Hz | Yes |
| Welch peak | 130.0 Hz | Yes |
| PLV at 125 Hz | 0.047 | Borderline |
| ITPC at 125 Hz | 0.183 | Yes |
| YIN frame-wise | NaN | **No (0/497 frames)** |

Expected range: 95–155 Hz.

**What the Results Mean**

YIN completely failed, i.e. 0 valid estimates out of 497 frames. To me this was the most surprising result of the whole project, and I'm not sure if this was a user error on my part since YIN is a widely-used pitch tracker in professional audio tools. Maybe the reason lies in that a drifting pitch creates a broader valley in YIN's difference function rather than a sharp dip, and the threshold is never crossed?

***Signal 4: Pure Noise***

```
n_trials = 10;
for tr = 1:n_trials
    noise4 = randn(N, 1);

    mE4            = run_RFB(noise4, fs, fmin_s, fmax_s);
    v4.cpei(tr)    = compute_cpei(mE4, period_hz, F0, cpei_bw);
    v4.plv(tr)     = compute_PLV(noise4, fs, F0);

    ep4 = zeros(n_epochs, epoch_len);
    for e = 1:n_epochs
        ep4(e,:) = noise4((e-1)*epoch_len+1 : e*epoch_len);
    end
    v4.itpc(tr)    = compute_ITPC(ep4, fs, F0);

    [p4,f4]        = pwelch(noise4, win_w, noverlap, nfft_w, fs, 'onesided');
    v4.wsnr(tr)    = welch_snr(p4, f4, F0, 5, noise_lo, noise_hi);

    [yf4, ~]       = yin_estimate(noise4, fs, yin_fmin, yin_fmax, ...
                                  yin_win, yin_thresh);
    v4.yin(tr)     = yf4;
end
```

Listing 12: Signal 4 generation

This signal is just pure random noise, so no periodic structure at all. I wrote this to check that none of the algorithms falsely claim to detect a signal when there is nothing there.

Table 4: Signal 4 results: Noise floor across 10 trials

| **Algorithm** | **Mean ± Std** | **False positives** |
|---|---|---|
| CPEI | $0.040 \pm 0.000$ | 0/10 |
| PLV | $0.006 \pm 0.004$ | 0/10 |
| ITPC | $0.089 \pm 0.003$ | 0/10 |
| Welch SNR | $0.22 \pm 0.77$ dB | 0/10 |
| YIN | NaN (no detections) | 0/10 |

**What the Results Mean**

All five algorithms produced zero false positives.

***Signal 5: Overlapping Harmonics***

```
F0_A = 120;   F0_B = 180;

sigA = zeros(N,1);
sigB = zeros(N,1);
for h = 1:n_harm
    sigA = sigA + (1/h)*sin(2*pi*h*F0_A*t);
    sigB = sigB + (1/h)*sin(2*pi*h*F0_B*t);
end
sigA   = sigA / max(abs(sigA));
sigB   = sigB / max(abs(sigB));
mixed5 = sigA + sigB + 0.3*randn(N,1);

% run RFB once, query CPEI separately for each speaker
mE5    = run_RFB(mixed5, fs, fmin_s, fmax_s);
cpei5A = compute_cpei(mE5, period_hz, F0_A, cpei_bw);
cpei5B = compute_cpei(mE5, period_hz, F0_B, cpei_bw);
```

Listing 13: Signal 5 generation

This signal mixes two simultaneous harmonic voices, A at 120 Hz and B at 180 Hz whose overtones overlap at 360, 720, and 1080 Hz.

Table 5: Signal 5 results: Two-speaker mixture detection

| **Algorithm** | **Value A** | **Value B** | **Det. A** | **Det. B** |
|---|---|---|---|---|
| CPEI | 0.158 | 0.061 | Yes | Yes |
| PLV | 0.478 | 0.464 | Yes | Yes |
| ITPC | 0.306 | 0.306 | Yes | Yes |
| Welch SNR (dB) | 29.08 | 29.67 | Yes | Yes |
| YIN | NaN | — | No | No |

**What the Results Mean**

All methods except YIN detected both speakers. The interesting difference is in that PLV, ITPC, and Welch gave nearly the same score for both speakers, while CPEI gave noticeably different scores (0.158 vs. 0.061). This asymmetry is meaningful since Speaker A at 120 Hz contributes more harmonics within the 50–500 Hz analysis range than Speaker B at 180 Hz, and CPEI captures that difference naturally through its energy normalization.

I believe YIN failed completely because the mixture of two harmonic series creates multiple competing dips in the difference function and the algorithm cannot choose between them.

## Final Algorithm Comparison

Table 6: Summary comparison of all five algorithms

| **Criterion** | **CPEI** | **Welch** | **PLV** | **ITPC** | **YIN** |
|---|---|---|---|---|---|
| Sensitivity (clean signal) | 0 dB | −10 dB | −10 dB | −10 dB | +20 dB only |
| Octave errors | 0/5 | 3/5 | N/A | N/A | 0/5[†] |
| Drifting pitch | ✓ | ✓ | Partial | Partial | ✗ |
| False pos. rate | Low | Low | Very low | Borderline | None |
| Two-speaker sep. | Asymm. | Symm. | Symm. | Symm. | ✗ |
| Requires known $F_0$ | No | No | Yes | Yes | No |
| Single scalar output | Yes | No | Yes | Yes | No |
| Catastrophic failures | None | Octave err. | Drifting | Drifting | Noise, drift |

[†] YIN avoids octave errors only when it manages to produce an estimate at all.

It seems that no single algorithm wins on every criterion, which makes sense. More specifically, PLV, ITPC, and Welch are more sensitive on clean signals. YIN is precise when conditions are ideal.

CPEI is the only method that handles all five conditions without completely breaking down in any of them. Its particular strengths lie in the zero octave errors and robustness to drifting pitch, which are exactly the two properties that matter most for measuring auditory neural entrainment in hearing-impaired listeners processing natural speech : )